

Методичка по теме: «Обучение на основе моделей в обучении с подкреплением»

Подробный разбор, учебный Python-пример с комментариями и 20 заданий среднего уровня

Материал ориентирован на студентов, изучающих основы обучения с подкреплением (reinforcement learning, RL), динамического программирования, планирования и поиска.

1. Цель методички

Цель данной методички — показать, что такое обучение на основе моделей, чем модель в RL отличается от «обычной» программы-симулятора, как агент может использовать модель для улучшения стратегии и почему идеи эвристического поиска естественно встраиваются в model-based RL.

После изучения материала студент должен уметь:

- объяснить различие между model-free и model-based подходами;
- формально задать модель среды через вероятности переходов и ожидаемые награды;
- понять, как агент оценивает и уточняет модель по данным взаимодействия со средой;
- реализовать простой алгоритм планирования по модели;
- использовать эвристическую идею приоритета обновлений состояний.

2. Ключевые понятия

Понятие	Смысл
Состояние s	Описание текущей ситуации среды.
Действие a	Выбор агента в состоянии s .
Награда r	Числовая оценка полезности результата действия.
Переход $P(s' s,a)$	Вероятность попасть в состояние s' из s после действия a .
Модель награды $R(s,a)$ или $R(s,a,s')$	Ожидаемая награда, связанная с переходом.
Стратегия $\pi(a s)$	Правило выбора действий.
Планирование	Улучшение оценок и стратегии с использованием модели, без новых реальных шагов в среде.
Эвристика	Дополнительное правило, позволяющее быстрее выбирать перспективные состояния или действия.

3. Что такое обучение на основе моделей

Обучение на основе моделей (model-based reinforcement learning) — это класс методов, в которых агент использует явную или неявную модель среды. Такая модель описывает, как

среда отвечает на действия агента: в какое следующее состояние она переходит и какую награду выдает.

В отличие от model-free методов, где агент обновляет ценности состояний и действий только по непосредственно наблюдаемым переходам, model-based методы добавляют этап планирования. Это означает, что после накопления опыта агент может «мысленно проигрывать» возможные переходы и дообучаться даже без дополнительного физического взаимодействия со средой.

Практически это дает три важных преимущества:

- более эффективное использование редких данных;
- возможность прогнозировать последствия действий до их реального выполнения;
- естественное соединение RL с поиском и классическим планированием.

4. Понятие модели в RL

В самом общем виде модель среды включает два компонента:

1. модель переходов: $P(s'|s,a)$;
2. модель наград: $R(s,a)$ или $R(s,a,s')$.

Если среда детерминированная, то для каждой пары (s,a) существует единственное следующее состояние s' и конкретная награда r . В стохастической среде возможны разные s' с разными вероятностями.

Модель может быть:

- известной заранее — например, если правила среды полностью заданы;
- изучаемой по данным — когда агент сам оценивает вероятности переходов и средние награды;
- приближенной — когда точная модель недоступна и строится статистическая аппроксимация.

5. Использование данных о модели

Когда у агента есть модель, данные о ней могут использоваться по крайней мере в четырех направлениях.

Способ использования	Что делает агент
Оценивание ценностей	Вычисляет ожидаемую полезность состояний и действий через уравнения Беллмана.
Планирование	Генерирует внутренние (воображаемые) переходы и обновляет оценки без реального опыта.
Выбор действий	Сравнивает ожидаемые последствия действий до выполнения.
Поиск	Разворачивает дерево состояний вперед и выбирает наиболее перспективные ветви.

Именно сочетание реального опыта и планирования делает model-based RL особенно сильным. Один реальный переход может породить много дополнительных обновлений, если модель уже накопила достаточно информации.

6. Эвристический поиск и его роль

Эвристический поиск — это поиск, в котором используются дополнительные оценки перспективности. В классическом AI эвристика помогает быстрее найти путь к цели, не перебирая все варианты одинаково подробно. В RL схожая идея применяется для того, чтобы уделять больше внимания важным состояниям или действиям.

В контексте model-based RL эвристики обычно используются так:

- сначала обновляются состояния, где недавно произошло большое изменение ценности;
- поиск разворачивается глубже в направлениях, обещающих высокую суммарную награду;
- редкие, но критически важные переходы получают повышенный приоритет;
- при ограниченном бюджете вычислений рассматриваются только наиболее перспективные узлы.

В нашей учебной реализации эвристика будет очень наглядной: мы будем хранить приоритеты состояний и в первую очередь выполнять планирование там, где ожидается наибольшее влияние на итоговую функцию ценности.

7. Учебная постановка задачи

Рассмотрим прямоугольный мир 5×5 . Агент стартует в левом верхнем углу и должен попасть в целевую клетку в правом нижнем углу. На каждом шаге доступны действия: вверх, вниз, влево и вправо. За обычный шаг агент получает награду -1 , а за попадание в цель — $+20$. Часть клеток является препятствиями.

Почему эта задача удобна для темы:

- модель переходов легко интерпретировать;
- видно различие между реальным опытом и планированием;
- можно показать, как эвристика ускоряет обучение;
- код остается компактным и понятным для студентов.

8. Идея алгоритма

Мы реализуем следующую схему:

3. Агент взаимодействует со средой и получает реальные переходы (s, a, r, s') .
4. По наблюдаемым переходам он строит модель: запоминает наиболее вероятное следующее состояние и среднюю награду для пары (s, a) .
5. После каждого реального шага агент выполняет несколько шагов планирования по модели.
6. Состояния для планирования выбираются не случайно, а по приоритету: чем сильнее недавно изменились оценки, тем выше шанс, что обновление этого состояния полезно.

По духу такая схема близка к Dyna-подходам и упрощенной идее prioritized sweeping. Для учебной цели она удобна тем, что сразу показывает роль модели и роль эвристики.

9. Полный Python-пример

Ниже приведен рабочий учебный код. Он написан максимально прозрачно, без внешних библиотек, кроме стандартного модуля random. Комментарии поясняют почти каждую логическую часть.

```
import random
from collections import defaultdict

# -----
# 1. ОПИСАНИЕ СРЕДЫ GRIDWORLD
# -----
class GridWorld:
    def __init__(self, width=5, height=5):
        # Размеры поля
        self.width = width
        self.height = height

        # Стартовое и целевое состояние
        self.start = (0, 0)
        self.goal = (4, 4)

        # Набор препятствий: в эти клетки заходить нельзя
        self.walls = {(1, 1), (1, 2), (3, 2)}

        # Все допустимые действия агента
        self.actions = ["up", "down", "left", "right"]

        # Текущее состояние среды
        self.state = self.start

    def reset(self):
        """Возврат среды к начальному состоянию."""
        self.state = self.start
        return self.state

    def step(self, action):
        """
        Выполнить действие и вернуть:
        next_state, reward, done
        """
        x, y = self.state

        # Вычисляем кандидат на следующее состояние
        if action == "up":
            candidate = (x, y - 1)
        elif action == "down":
            candidate = (x, y + 1)
        elif action == "left":
            candidate = (x - 1, y)
        else:
            candidate = (x + 1, y)

        # Проверяем границы поля и препятствия.
        # Если переход недопустим, агент остается на месте.
        nx, ny = candidate
```

```

        if nx < 0 or nx >= self.width or ny < 0 or ny >= self.height or
candidate in self.walls:
            candidate = self.state

        self.state = candidate

        # Награда за обычный шаг
        reward = -1
        done = False

        # Если достигнута цель, завершаем эпизод
        if self.state == self.goal:
            reward = 20
            done = True

        return self.state, reward, done

# -----
# 2. АГЕНТ С МОДЕЛЬЮ СРЕДЫ И ПЛАНИРОВАНИЕМ
# -----
class ModelBasedAgent:
    def __init__(self, actions, alpha=0.1, gamma=0.95, epsilon=0.2,
planning_steps=15):
        self.actions = actions

        # Параметры обучения
        self.alpha = alpha          # скорость обучения
        self.gamma = gamma          # коэффициент дисконтирования
        self.epsilon = epsilon      # вероятность случайного действия
        self.planning_steps = planning_steps # сколько шагов планирования
        # делать после реального шага

        # Q-таблица: оценка полезности действия a в состоянии s
        self.Q = defaultdict(float)

        # МОДЕЛЬ СРЕДЫ:
        # model[(s, a)] = (predicted_next_state, predicted_reward)
        self.model = {}

        # Счетчики посещений для усреднения наград
        self.reward_sum = defaultdict(float)
        self.reward_count = defaultdict(int)

        # Предшественники для эвристического поиска:
        # predecessors[s_next] = set of (s, a)
        # Нужно, чтобы после изменения ценности состояния
        # можно было быстро понять, какие состояния "зависят" от него.
        self.predecessors = defaultdict(set)

        # Список пар (state, action) с приоритетами.
        # Для простоты вместо настоящей очереди с приоритетами
        # используем словарь: pair -> priority.
        self.priority = defaultdict(float)

    def get_q(self, state, action):
        """Получить значение Q(s, a)."""
        return self.Q[(state, action)]

    def best_action_value(self, state):
        """Максимальное значение Q по всем действиям в состоянии state."""
        return max(self.get_q(state, a) for a in self.actions)

```

```

def choose_action(self, state):
    """
    epsilon-greedy выбор действия:
    иногда исследуем случайно,
    иначе берем действие с максимальным Q.
    """
    if random.random() < self.epsilon:
        return random.choice(self.actions)

    values = [self.get_q(state, a) for a in self.actions]
    max_value = max(values)

    # Может быть несколько лучших действий - выбираем случайно среди них
    best_actions = [a for a in self.actions if self.get_q(state, a) ==
max_value]
    return random.choice(best_actions)

def update_from_real_experience(self, state, action, reward, next_state):
    """
    Обычное Q-learning обновление на реальном опыте.
    """
    old_q = self.get_q(state, action)
    target = reward + self.gamma * self.best_action_value(next_state)
    self.Q[(state, action)] = old_q + self.alpha * (target - old_q)

    # Вернем абсолютную величину изменения – это пригодится
    # как простая эвристика важности обновления.
    return abs(target - old_q)

def update_model(self, state, action, reward, next_state):
    """
    Обновить внутреннюю модель среды.
    Здесь модель детерминированная:
    для пары (s, a) храним наиболее свежий next_state
    и среднюю награду.
    """
    key = (state, action)

    # Обновляем среднюю награду
    self.reward_sum[key] += reward
    self.reward_count[key] += 1
    avg_reward = self.reward_sum[key] / self.reward_count[key]

    # Запоминаем прогноз модели
    self.model[key] = (next_state, avg_reward)

    # Запоминаем, что состояние next_state может быть
    # достигнуто из (state, action)
    self.predecessors[next_state].add((state, action))

def push_priority(self, state, action, amount):
    """
    Повысить приоритет пары (state, action).
    Чем больше amount, тем раньше эту пару
    стоит рассмотреть в фазе планирования.
    """
    if amount > self.priority[(state, action)]:
        self.priority[(state, action)] = amount

def planning_step(self):
    """
    Один шаг планирования по модели.
    Выбираем пару (s, a) с максимальным приоритетом,

```

```

        затем обновляем Q как будто этот переход реально произошел.
        """
        if not self.priority:
            return

        # Находим пару с максимальным приоритетом
        state_action = max(self.priority, key=self.priority.get)
        max_priority = self.priority[state_action]

        # Если приоритет слишком мал, смысла обновлять почти нет
        if max_priority < 1e-8:
            return

        # Удаляем выбранную пару из очереди приоритетов
        del self.priority[state_action]

        state, action = state_action

        # Если модели еще нет, планировать нельзя
        if (state, action) not in self.model:
            return

        next_state, predicted_reward = self.model[(state, action)]

        # "Воображаемое" обновление Q по модели
        old_q = self.get_q(state, action)
        target = predicted_reward + self.gamma *
self.best_action_value(next_state)
        self.Q[(state, action)] = old_q + self.alpha * (target - old_q)

        # После обновления состояния next_state может измениться важность его
        предшественников.
        # Поэтому поднимаем приоритеты для всех (s_prev, a_prev), ведущих в
        state.
        for prev_state, prev_action in self.predecessors[state]:
            if (prev_state, prev_action) in self.model:
                model_next_state, model_reward = self.model[(prev_state,
prev_action)]
                estimate = model_reward + self.gamma *
self.best_action_value(model_next_state)
                diff = abs(estimate - self.get_q(prev_state, prev_action))
                self.push_priority(prev_state, prev_action, diff)

    def train_episode(self, env, max_steps=200):
        """
        Один эпизод взаимодействия агента со средой.
        """
        state = env.reset()
        total_reward = 0

        for _ in range(max_steps):
            # 1. Выбираем действие
            action = self.choose_action(state)

            # 2. Получаем реальный переход
            next_state, reward, done = env.step(action)
            total_reward += reward

            # 3. Обновляем Q по реальному опыту
            change = self.update_from_real_experience(state, action, reward,
next_state)

            # 4. Обновляем модель среды

```

```

        self.update_model(state, action, reward, next_state)

        # 5. Помещаем текущую пару в приоритетную "очередь"
        self.push_priority(state, action, change)

        # 6. Выполняем несколько шагов планирования по модели
        for _ in range(self.planning_steps):
            self.planning_step()

        state = next_state

        if done:
            break

    return total_reward

# -----
# 3. ЗАПУСК ОБУЧЕНИЯ
# -----
if __name__ == "__main__":
    random.seed(42)

    env = GridWorld()
    agent = ModelBasedAgent(
        actions=env.actions,
        alpha=0.1,
        gamma=0.95,
        epsilon=0.2,
        planning_steps=15
    )

    episodes = 120
    rewards = []

    for episode in range(episodes):
        ep_reward = agent.train_episode(env)
        rewards.append(ep_reward)

        # Печатаем прогресс каждые 10 эпизодов
        if (episode + 1) % 10 == 0:
            avg_last_10 = sum(rewards[-10:]) / 10
            print(f"Эпизод {episode + 1:3d} | средняя награда за последние 10 = {avg_last_10:6.2f}")

    # Покажем найденную стратегию в каждой клетке
    print("\nПриближенная стратегия:")
    for y in range(env.height):
        row = []
        for x in range(env.width):
            state = (x, y)

            if state in env.walls:
                row.append("####")
            elif state == env.goal:
                row.append("GOAL")
            else:
                best_a = max(env.actions, key=lambda a: agent.get_q(state, a))
                row.append(best_a[:4])
        print(" | ".join(row))

```


10. Подробный разбор программы

10.1. Класс среды GridWorld

Среда реализует интерфейс `reset/step`. Для учебной цели важно, что переходы легко проследить вручную. Если агент пытается выйти за границу или попасть в стену, он остается на месте. Это полезно, потому что у модели появляется повторяемая структура переходов.

10.2. Q-таблица

Хотя тема посвящена модели, сама оценка полезности действий хранится в виде $Q(s,a)$. Это удобно: реальные и воображаемые обновления можно выполнять одной и той же формулой.

10.3. Модель среды

В словаре `model` для каждой пары (s,a) хранится прогноз следующего состояния и средней награды. В более сложных задачах вместо одного состояния можно хранить распределение вероятностей перехода.

10.4. Обновление по реальному опыту

Функция `update_from_real_experience` выполняет стандартное обновление Q-learning. Это важно: модель не заменяет опыт, а усиливает его.

10.5. Фаза планирования

После каждого реального шага агент несколько раз вызывает `planning_step`. Эти шаги не требуют обращения к среде. Именно здесь проявляется смысл `model-based RL`: один реальный шаг порождает серию дополнительных вычислительных обновлений.

10.6. Эвристический приоритет

Мы оцениваем важность пары (s,a) величиной изменения $target - Q$. Если расхождение большое, значит обновление потенциально меняет оценку заметно, и его разумно обработать раньше. Это и есть эвристическая идея приоритизации.

10.7. Предшественники состояния

Если ценность состояния изменилась, полезно быстро найти те состояния и действия, которые ведут в него. Для этого ведется структура `predecessors`. В полном `prioritized sweeping` именно эта идея резко ускоряет распространение полезной информации назад по графу состояний.

11. Что студент должен увидеть после запуска

В начале обучения средняя награда обычно низкая: агент много блуждает, сталкивается с препятствиями и часто делает лишние шаги. Затем, по мере накопления модели и роста

полезности планирования, траектории становятся короче и стабильнее. Итоговая политика должна направлять агента к цели, обходя стены.

Важно подчеркнуть, что улучшение происходит не только за счет новых эпизодов, но и за счет внутренних обновлений по модели. Если уменьшить `planning_steps` до нуля, агент станет фактически `model-free` и, как правило, будет обучаться медленнее.

12. Как этот пример связан с теорией

Теоретическая идея	Где проявляется в коде
Модель переходов и наград	<code>self.model[(state, action)] = (next_state, avg_reward)</code>
Использование модели для планирования	метод <code>planning_step</code>
Разделение реального и внутреннего опыта	<code>train_episode</code> : сначала <code>step</code> среды, затем <code>planning_step</code>
Эвристический поиск	словарь <code>priority</code> и выбор пары с максимальным приоритетом
Распространение влияния назад	структура <code>predecessors</code>

13. Возможные упрощения и расширения

- Упрощение: убрать эвристику и планировать случайные пары (s,a). Тогда получится более простой Дупа-подобный вариант.
- Расширение: сделать среду стохастической, чтобы одно и то же действие иногда вело в разные клетки.
- Расширение: хранить не одно следующее состояние, а оценки вероятностей переходов.
- Расширение: заменить таблицу Q на приближение функцией, если пространство состояний велико.
- Расширение: сравнить скорость обучения при разных значениях `planning_steps`.

14. Типичные ошибки

7. Смешивание понятий «модель» и «стратегия». Модель описывает среду, стратегия — поведение агента.
8. Забытое дисконтирование γ при вычислении `target`.
9. Использование планирования до того, как в модели вообще появились данные о переходах.
10. Отсутствие структуры `predecessors`, из-за чего идея приоритетного распространения эффекта назад исчезает.
11. Слишком большой `epsilon`, из-за чего даже хорошая стратегия долго не проявляется на практике.

15. 20 заданий среднего уровня

1. Измените размеры мира на 7×7 и подберите новую конфигурацию препятствий. Сравните, как меняется средняя награда по эпизодам.

2. Уменьшите `planning_steps` до 0, 5 и 30. Для каждого случая проведите серию запусков и сравните скорость обучения.
3. Сделайте шаговую награду равной -0.1 вместо -1 . Объясните, как это влияет на длину найденных траекторий.
4. Добавьте вторую целевую клетку с меньшей наградой. Исследуйте, в каких условиях агент предпочтет ближнюю цель, а в каких — дальнюю.
5. Сделайте одну «опасную» клетку со штрафом -10 . Изучите, научится ли агент обходить ее заранее.
6. Реализуйте стохастическое действие: с вероятностью 0.2 агент движется в случайную сторону. Измените модель так, чтобы она хранила частоты переходов.
7. Переделайте модель награды: храните не среднюю награду, а отдельные статистики по каждому возможному `next_state`.
8. Сравните стратегию `epsilon-greedy` со стратегией `softmax`-выбора действий.
9. Добавьте ограничение на длину эпизода и исследуйте, как оно влияет на обучение и качество модели.
10. Реализуйте визуальный вывод траектории агента в консоль после завершения обучения.
11. Сделайте случайную инициализацию стартового состояния среди нескольких допустимых клеток. Проверьте, насколько устойчиво обучается агент.
12. Добавьте затухание `epsilon` по эпизодам. Подберите график уменьшения и сравните результат с постоянным `epsilon`.
13. Уберите эвристический приоритет и выбирайте пары для планирования случайно. Сравните число эпизодов до стабилизации политики.
14. Реализуйте настоящую очередь с приоритетом (`heapq`) вместо словаря. Сравните удобство и скорость работы.
15. Измените критерий приоритета: используйте не $|\text{target} - Q|$, а модуль TD-ошибки, умноженный на число посещений состояния. Проанализируйте различие.
16. Постройте таблицу значений $V(s) = \max_a Q(s, a)$ после обучения и интерпретируйте, какие зоны мира считаются агентом наиболее перспективными.
17. Добавьте телепорт: из одной клетки агент мгновенно переносится в другую. Опишите, как это отражается в модели переходов.
18. Реализуйте частично известную модель: часть переходов задайте заранее, а часть дооценивайте по данным. Сравните скорость обучения с полностью неизвестной моделью.
19. Измените постановку так, чтобы цель перемещалась каждые несколько эпизодов. Исследуйте, насколько быстро агент адаптируется к уже накопленной моделию.
20. Подготовьте мини-отчет: опишите среду, параметры, выбранную эвристику, график наград и вывод о полезности `model-based` компонента.

16. Рекомендации по оформлению отчета

- кратко описать среду и цель управления;
- привести основные параметры: α , γ , ϵ , число шагов планирования;
- объяснить, как хранится модель;
- показать, какая эвристика использовалась;
- приложить таблицу или график средней награды по эпизодам;
- сделать вывод о влиянии планирования и эвристики на скорость обучения.

17. Контрольные вопросы

12. Почему наличие модели позволяет использовать один реальный опыт эффективнее?
13. Чем детерминированная модель отличается от стохастической?
14. Зачем в алгоритме нужны предшественники состояния?
15. Почему эвристический выбор обновлений может ускорять обучение?
16. В чем различие между поиском по модели и простым исполнением стратегии в среде?
17. Почему в больших задачах точную модель часто заменяют приближенной?

18. Вывод

Обучение на основе моделей занимает важное место в RL, потому что соединяет обучение по опыту с прогнозированием и планированием. Как только агент получил хотя бы приближенную модель переходов и наград, он может многократно переиспользовать уже наблюдаемые данные. Добавление эвристического поиска позволяет распределять вычислительные ресурсы неравномерно, а значит, ускорять распространение полезной информации по пространству состояний. Именно поэтому model-based RL является естественным мостом между классическим поиском, динамическим программированием и современными методами обучения с подкреплением.